

BAB IV

PEMBUATAN LARAVEL BREEZE

3.1 Landasan Teori

Laravel Breeze adalah starter kit resmi Laravel yang menyediakan implementasi otentikasi dasar yang ringan dan mudah dipahami. Breeze menggunakan Blade sebagai templating engine dan Tailwind CSS untuk styling, sehingga cocok untuk pengembangan aplikasi berbasis server-side rendering (SSR) yang sederhana namun fleksibel.

Dengan Laravel Breeze, developer langsung mendapatkan fitur-fitur penting seperti:

- Registrasi dan login pengguna
- Verifikasi email
- Reset password via email
- Logout dan manajemen sesi dasar
- Middleware verified untuk membatasi akses hanya bagi pengguna yang telah memverifikasi email

Struktur kode Breeze sangat minimalis dan mudah dimodifikasi, menjadikannya pilihan ideal bagi developer yang ingin kontrol penuh terhadap alur otentikasi dan tampilan antarmuka. Breeze juga mendukung ekspansi ke stack lain seperti Vue atau React jika dibutuhkan, namun tetap mempertahankan kesederhanaan sebagai nilai utama.

Karena tidak menyertakan fitur lanjutan seperti manajemen tim, two-factor authentication (2FA), atau manajemen perangkat seperti Jetstream, Laravel Breeze sangat cocok untuk proyek yang mengutamakan kesederhanaan, kecepatan pengembangan, dan fleksibilitas kustomisasi

3.2 Tujuan

1. Mahasiswa memahami implementasi fitur otentikasi dan manajemen pengguna menggunakan Laravel Breeze
2. Mahasiswa mampu mengelola data dan sesi perangkat dengan Breeze.

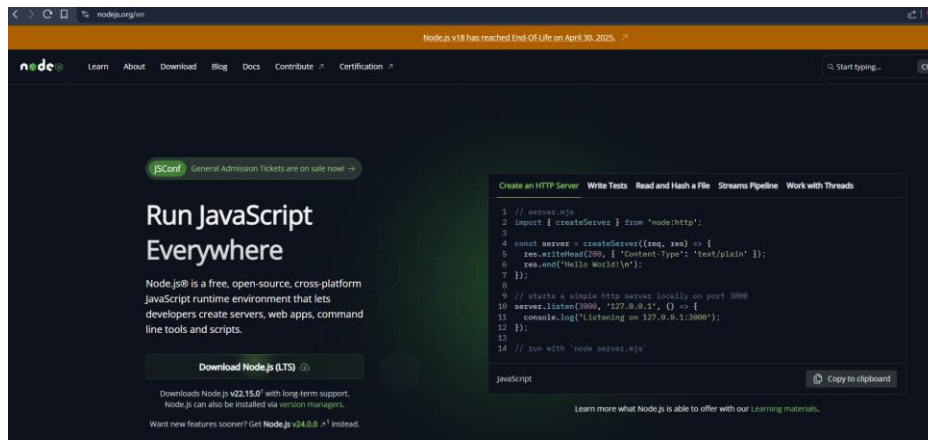
3.3 Laravel Jetstream

3.3.1 Alat dan Bahan

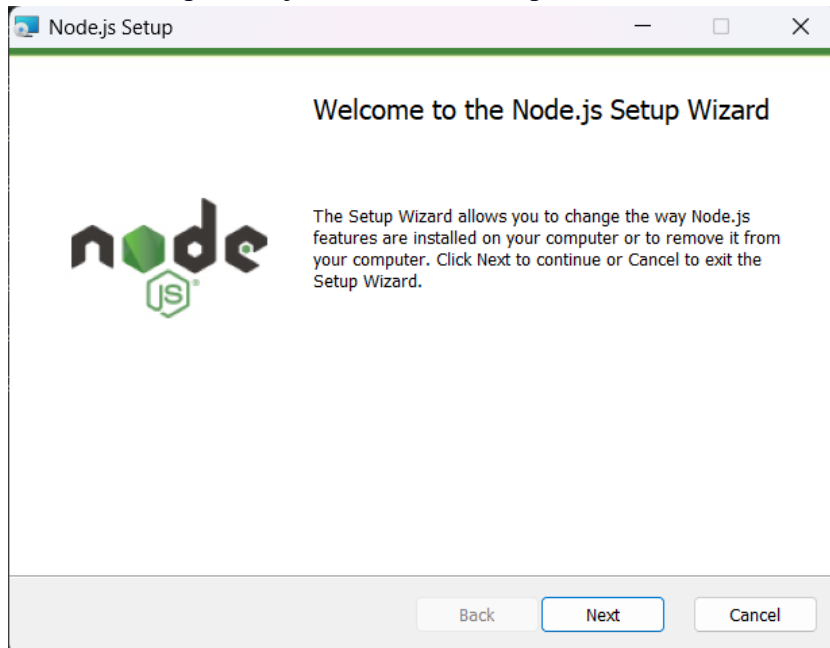
1. Node.js
2. Project Laravel sebelumnya
3. Visual Basic Code

3.3.2 Pembuatan Laravel Jetstream

1. Unduh Node.js yang versi terbaru



2. Jalankan Setup Node.js dan lakukan sampai selesai



3. Setelah menginstall Node.js, buka project Laravel melalui Visual Basic Code
4. Buka terminal dan masukan *Command* berikut “composer require laravel/Breeze”


```
$ composer require laravel/breeze
```
5. Selanjutnya masukan *Command* berikut “php artisan breeze:install”


```
php artisan breeze:install
```
6. Lalu masukan *Command* berikut
“npm install”
“npm run dev”
7. Untuk memastikan semua package sudah terinstall dapat dengan *Command* berikut
“composer update”
8. Setelah menginstall semuanya jalankan Kembali server dengan *Command* berikut “php artisan serve”
9. Jika tampilannya sudah seperti berikut berarti berhasil.

[Log in](#)

[Register](#)

Let's get started

Laravel has an incredibly rich ecosystem. We suggest starting with the following.

- Read the [Documentation](#)
- Watch video tutorials at [Laracasts](#)

[Deploy now](#)



Email

Password

☐ Remember me

[Forgot your password?](#)

[LOG IN](#)



Name

Email

Password

Confirm Password

[Already registered?](#)

[REGISTER](#)

3.4 Tugas

1. Buat Akun dengan fitur bawaan Breeze.
2. Sambungkan project CRUD sebelumnya dengan Laravel Breeze dan **DIWAJIBKAN** login terlebih dahulu.

BAB V

PEMBUATAN MAILTRAP

5.1 Landasan Teori

Mailtrap adalah layanan SMTP yang dirancang khusus untuk menguji pengiriman email dalam lingkungan pengembangan. Dengan Mailtrap, developer dapat memastikan bahwa fitur email dalam aplikasi Laravel berjalan dengan baik tanpa risiko mengirim email ke pengguna nyata selama tahap testing.

Dalam Laravel, Mailtrap digunakan untuk:

- **Verifikasi Email:** Saat pengguna mendaftar, Laravel dapat mengirimkan tautan verifikasi ke alamat email mereka melalui Mailtrap. Developer dapat memantau email tersebut di dashboard Mailtrap.
- **Reset Password:** Fitur lupa password mengirimkan tautan reset ke email pengguna, yang dapat diuji melalui Mailtrap.
- **Notifikasi Sistem:** Email otomatis seperti konfirmasi peminjaman buku atau pengembalian dapat dikirim ke Mailtrap untuk diuji sebelum digunakan secara live.

5.2 Tujuan

1. Mahasiswa dapat memahami konsep dasar pengujian email dalam pengembangan aplikasi web dan pentingnya verifikasi email untuk keamanan pengguna.
2. Mahasiswa mampu mengimplementasikan layanan Mailtrap dalam proyek Laravel untuk menguji fitur pengiriman email seperti verifikasi akun dan reset password secara aman dan efisien

5.3 Pembuatan Mailtrap

1. Pembuatan Akun
2. Masuk ke mailtrap.io, lalu registrasi

Create Free Account

By creating a Mailtrap account, you agree to our
[Terms of Service and Privacy Policy](#)

 Use Google account

 Use Github account

 Use Office 365 account

OR

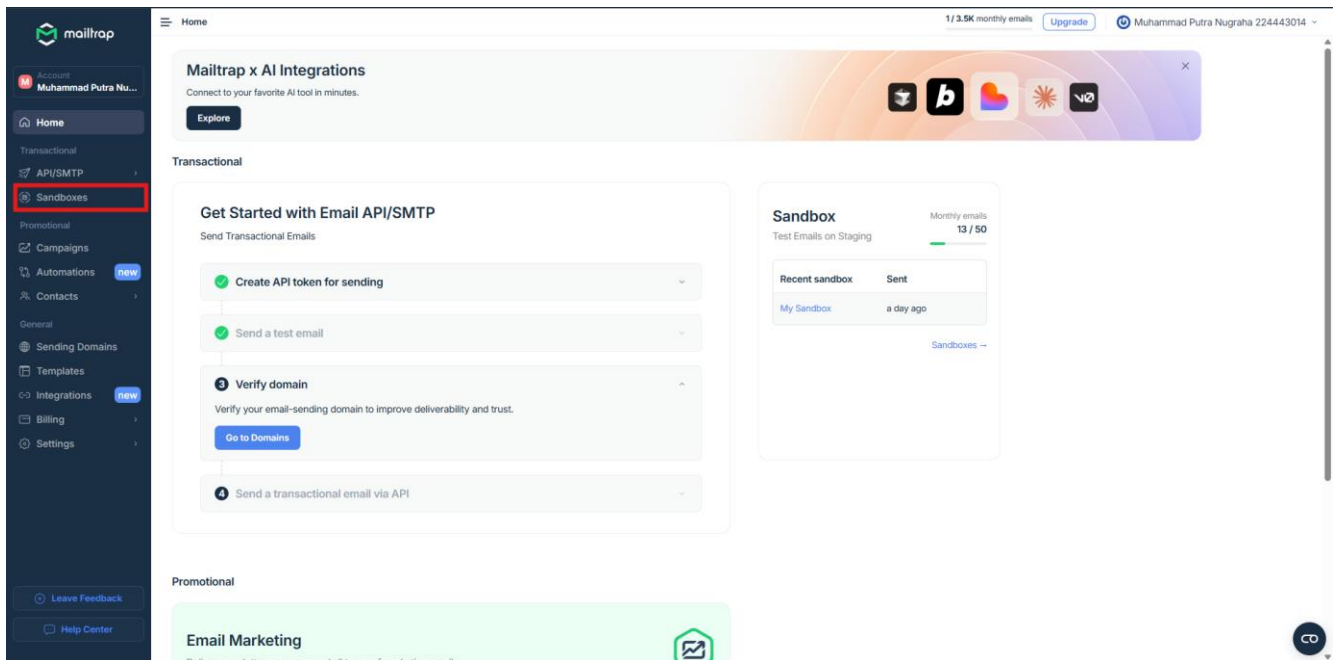
Sign Up With Your Email

We'll occasionally send you account related emails.



 No credit card needed

3. Setelah memasuki dashboard, lalu click sandboxes



4. Lalu click my sandbox

My Project :						Add Sandbox
Sandboxes	Total Sent	Messages	Max size	Last message	Action	
My Sandbox	13	5 / 10	10	a day ago	Settings	Delete

5. Lalu Masukkan Credentials ini ke .env kalian

Host

sandbox.smtp.mailtrap.io

Port

25, 465, 587 or 2525

Username

[REDACTED]

Password

[REDACTED]

Auth

PLAIN, LOGIN and CRAM-MD5

TLS

Optional (STARTTLS on all ports)

Code Samples

cURL

PHP: Laravel 9+

Node.js

Python

C#

Ruby

Other

Copy

```

1 # Looking to send emails in production? Check out our Email API/SMTP product!
2 MAIL_MAILER=smtp
3 MAIL_HOST=sandbox.smtp.mailtrap.io
4 MAIL_PORT=2525
5 MAIL_USERNAME=[REDACTED]
6 MAIL_PASSWORD=[REDACTED]

```

Laravel provides a clean, simple API over the popular Symfony Mailer library. With the default Laravel setup you can configure your mailing configuration by setting these values in the .env file in the root directory of your project.

6. Konfigurasi .env mailtrap

```
MAIL_MAILER=smtp
MAIL_HOST=sandbox.smtp.mailtrap.io
```

```
MAIL_PORT=2525
MAIL_USERNAME=your_mailtrap_username
MAIL_PASSWORD=your_mailtrap_password
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS="laravel@example.com"
MAIL_FROM_NAME="${APP_NAME}"
```

7. Pastikan model User mengimplementasi `MustVerifyEmail`:

```
// app/Models/User.php
use Illuminate\Contracts\Auth\MustVerifyEmail;

class User extends Authenticatable implements MustVerifyEmail
{
    use HasFactory, Notifiable;
}
```

8. Tambahkan middleware `verified` pada route dashboard:

```
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');
```

9. Tambahkan package spatie/laravel-permission:

```
composer require spatie/laravel-permission
```

10. Selanjutnya lakukan publikasi konfigurasi dan Migrasi package dengan Command berikut:

```
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
```

11. Setelah itu jalankan Migrasi dengan Command berikut:

```
php artisan migrate
```

12. Update file "User.php" yang berada pada "app/Models/User.php", dengan memasukan code berikut:

```
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable implements MustVerifyEmail {
    use HasFactory, Notifiable, HasRoles;
}
```

13. Untuk mengimplementasikan middleware untuk mengatur hak akses pada file "web.php" yang berada pada file "routes/web.php", Update file tersebut dengan code berikut:

```

<?php

use App\Http\Controllers\ArticleController;
use App\Http\Controllers\CategoryController;
use Illuminate\Support\Facades\Route;

// Route akses dasar dan dashboard
Route::get('/', function () {
    return view('welcome');
});
Route::middleware([
    'auth:sanctum',
    config('jetstream.auth_session'),
    'verified',
])->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    }->name('dashboard'));
});

// Route dengan auth umum dan permission 'manage categories' untuk kategori
Route::middleware(['auth', 'role_or_permission:manage categories'])->group(function () {
    // Kategori hanya bisa diakses oleh user yang memiliki permission 'manage categories'
    Route::resource('categories', CategoryController::class);

    // Route artikel yang tidak memerlukan permission khusus
    Route::get('articles', [ArticleController::class, 'index'])->name('articles.index');
    Route::get('articles/create', [ArticleController::class, 'create'])->name('articles.create');
    Route::post('articles', [ArticleController::class, 'store'])->name('articles.store');
});

// Akses edit artikel hanya untuk user dengan permission "edit articles"
Route::middleware('permission:edit articles')->group(function () {
    Route::get('articles/{article}/edit', [ArticleController::class, 'edit'])->name('articles.edit');
    Route::put('articles/{article}', [ArticleController::class, 'update'])->name('articles.update');
});

// Akses hapus artikel hanya untuk user dengan permission "delete articles"
Route::middleware('permission:delete articles')->group(function () {
    Route::delete('articles/{article}', [ArticleController::class, 'destroy'])->name('articles.destroy');
});

```


14. Selanjutnya edit pada file "app.php" yang berada pada file "bootstrap/app.php", update berikut dengan code berikut:

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->alias([
            'role' => \Spatie\Permission\Middleware\RoleMiddleware::class,
            'permission' =>
\Spatie\Permission\Middleware\PermissionMiddleware::class,
            'role_or_permission' =>
\Spatie\Permission\Middleware\RoleOrPermissionMiddleware::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

15. Setelah membuat middleware untuk website, selanjutnya adalah mengelola role dan permission dengan package, dengan cara menggunakan tinker pada terminal.

```
php artisan tinker
use Spatie\Permission\Models\Permission;
$permission1 = Permission::create(['name' => 'edit articles']);
$permission2 = Permission::create(['name' => 'delete articles']);
$permission3 = Permission::create(['name' => 'user management']);
```

16. Untuk membuat role dapat dengan memasukan Command berikut

```
use Spatie\Permission\Models\Role;
$role1 = Role::create(['name' => 'writer']);
$role2 = Role::create(['name' => 'admin']);
```

17. Untuk cara menambahkan dan memverifikasi permission dalam aplikasi dengan menambahkan permission pada role, dapat memasukan Command berikut:

```
$role1->givePermissionTo(['edit articles', 'delete articles']);

$role2->givePermissionTo(['edit articles', 'delete articles', 'user management']);
```

18. Untuk cara menugaskan Role ke User dapat memasukan Command berikut:

```
use App\Models\User;
$user = User::where('email', 'email_admin')->first();
$user->assignRole('admin');

$user2 = User::where('email', '=', 'email_writer')->first();
$user2->assignRole('writer');
```

19. Jalankan migrasi

```
php artisan migrate
```

20. Jalankan server development

```
php artisan serve
```

21. Akses aplikasi

Buka browser dan kunjungi `http://127.0.0.1:8000`

22. Dokumentasi Akhir Cek Mailtrap

The screenshot displays the Mailtrap interface for an email verification process. On the left, a sidebar lists multiple 'Verify Email Address' emails sent to various addresses, with the most recent one selected. The main panel shows the content of the selected email. The email is from 'Sistem Perpustakaan' and is addressed to '<2@gmail.com>'. The body of the email includes a 'Hello!' greeting, a request to click a button to verify the email address, a 'Verify Email Address' button, and a message stating that no further action is required if the user did not create an account. The email is signed 'Regards, Laravel'. At the bottom, there is a long URL for verification: <http://127.0.0.1:8000/verify-email/7/4b51e227fe6c295d9b2b434d91649226e8ad720a?expires=1762580746&signature=c2a2f3e0d06a5534cfff66e516804a68dbd1e4ed55e278c66775503a9b31392>. The footer of the email indicates '© 2025 Laravel. All rights reserved.'

5.4 Tugas

Sambungkanlah verifikasi registrasi mailtrap ini ke project CRUD sebelumnya yang memakai laravel breeze